1.Insert at Beginning

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void beginning() {
    struct node *first,*second,*tail;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    tail = (struct node*)malloc(sizeof(struct node));

    first->data = 20;
    second->data = 30;
    tail->data = 40;

    head = first;

    first->next = second;
    second->next = tail;
    tail->next = head;
```

Output

```
list:10 20 30 40

=== Code Execution Successful ===
```

```c
    tail->next = head;
}

void insertbeginning(int data){
    struct node *newnode,*tail;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

    tail = head;

    while(tail->next != head){
        tail = tail->next;
    }

    newnode->next = head;
    tail->next = newnode;

    head = newnode;
}

void display(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
```

```
list:10 20 30 40

=== Code Execution Succ
```

main.c

```c
void display(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertbeginning(10);

    printf("list:");
    display();

    return 0;
}
```

Output

```
list:10 20 30 40

=== Code Execution Successful ===
```

2.Insert at End

main.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void beginning() {
    struct node *first,*second,*tail;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    tail = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    tail->data = 30;

    head = first;

    first->next = second;
    second->next = tail;
    tail->next = head;
```

Output

```
List: 10 20 30 40

=== Code Execution Successful ===
```

```c
}

void insertend(int data){
    struct node *newnode,*temp;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

    temp = head;

    while(temp->next != head){
        temp = temp->next;
    }

    temp->next = newnode;
    newnode->next = head;
}

void display(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }
```

```
List: 10 20 30 40

=== Code Execution Successful ===
```

main.c

```c
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertend(40);

    printf("List: ");
    display();

    return 0;
}
```

```
List: 10 20 30 40

=== Code Execution Succe
```

3.Insert at position

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void beginning() {
    struct node *first,*second,*tail;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    tail = (struct node*)malloc(sizeof(struct node));

    first->data = 20;
    second->data = 30;
    tail->data = 40;

    head = first;

    first->next = second;
    second->next = tail;
    tail->next = head;
```

```
List: 20 25 30 40

=== Code Execution Successful ===
```

```c
}

void insertpos(int data,int pos){
    struct node *newnode,*temp;
    int i;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

    if(pos == 1){
        temp = head;

        while(temp->next != head){
            temp = temp->next;
        }

        newnode->next = head;
        temp->next = newnode;
        head = newnode;
        return;
    }

    temp = head;
```

```
List: 20 25 30 40

=== Code Execution Succe
```

main.c

```c

    for(i = 1; i < pos-1; i++){
        temp = temp->next;
    }

    newnode->next = temp->next;
    temp->next = newnode;
}

void display(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
```

Output

```
List: 20 25 30 40

=== Code Execution Successful ===
```

main.c

```c
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();

    insertpos(25,2);

    printf("List: ");
    display();

    return 0;
```

Output

```
List: 20 25 30 40

=== Code Execution Successf
```

4.Deletion at beginning

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void beginning(){
    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;

    first->next = second;
    second->next = third;
    third->next = head;
```

Output

```
List: 20 30

=== Code Execution Successful ===
```

```c

void deletebegin(){
    struct node *temp,*tail;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    temp = head;
    tail = head;

    while(tail->next != head){
        tail = tail->next;
    }

    head = head->next;
    tail->next = head;

    free(temp);
}

void display(){
    struct node *temp = head;

    if(head == NULL){
```

```
List: 20 30

=== Code Execution Successful ===
```

```c
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ",temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){

    beginning();
    deletebegin();

    printf("List: ");
    display();

    return 0;
}
```

```
List: 20 30

=== Code Execution Successful ===
```

5.Deletion at End

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void beginning(){
    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;

    first->next = second;
    second->next = third;
    third->next = head;
```

```
List: 10 20

=== Code Execution Successful ==
```

```c

void deleteend(){
    struct node *temp,*prev;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    temp = head;

    while(temp->next != head){
        prev = temp;
        temp = temp->next;
    }

    prev->next = head;

    free(temp);
}

void display(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
```

```
List: 10 20

=== Code Execution Successful =
```

```c

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ",temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){

    beginning();

    deleteend();

    printf("List: ");
    display();

    return 0;
}
```

```
List: 10 20

=== Code Execution Success
```

6.Deletion at position

main.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *next;
};

struct node *head = NULL;

void beginning(){
    struct node *first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 10;
    second->data = 20;
    third->data = 30;

    head = first;

    first->next = second;
    second->next = third;
    third->next = head;
```

Output

```
List: 10 30

=== Code Execution Successful ===
```

main.c

```c
    third->next = head;
}

void deletepos(int pos){
    struct node *temp,*del;
    int i;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    if(pos == 1){
        struct node *tail = head;

        while(tail->next != head){
            tail = tail->next;
        }

        del = head;
        head = head->next;
        tail->next = head;

        free(del);
        return;
    }
```

Output

```
List: 10 30

=== Code Execution Successful ===
```

```c
        return;
    }

    temp = head;

    for(i=1;i<pos-1;i++){
        temp = temp->next;
    }

    del = temp->next;
    temp->next = del->next;

    free(del);
}

void display(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ",temp->data);
    temp = temp->next;

```

```
List: 10 30

=== Code Execution Successful
```

```c

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ",temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}

int main(){

    beginning();

    deletepos(2);

    printf("List: ");
    display();

    return 0;
}
```

```
List: 10 30

=== Code Execution Successful ===
```